

1. Nature of the Project: Evaluation, Not Training

This project is designed as a model-evaluation and auditing pipeline, not a traditional supervised machine learning system trained on labeled data and periodically retrained. The core objective is to evaluate the behavior of pre-trained Large Language Models (LLMs) using adversarial prompts, judge models, and bias/failure analysis.

The models used in this system (e.g., LLaMA 3.1) are:

- Externally hosted
- Pre-trained by third-party providers
- Not fine-tuned or retrained inside this pipeline

Therefore, there is no internal training loop that can be retriggered when drift or decay is detected.

2. Why Traditional Model Decay Monitoring Does Not Apply

Monitoring for model decay (accuracy, F1, precision, recall) assumes:

- A fixed training dataset
- A retrainable model
- A predictive task with ground-truth labels

In this project:

- The task is LLM response generation + safety evaluation
- There is no ground-truth classification/regression label
- The outputs are evaluated using:
 - A judge LLM
 - Bias detection
 - Failure detection
 - Safety and relevance metrics

Hence, traditional performance decay metrics such as accuracy, F1-score, or recall are not meaningful for this task.

3. Why Data Drift Detection Does Not Trigger Retraining

Tools such as Evidently AI or TensorFlow Data Validation are designed to detect:

- Feature distribution changes in numerical or categorical structured data
- Drift relative to a training dataset

In this project:

- The “data” consists of prompts and adversarial test cases
- The objective is to vary distributions to test robustness intentionally
- No training dataset baseline requires protection from drift

Therefore, data drift is expected and intentional, not a failure condition.

4. Why Automated Retraining Is Architecturally Incorrect Here

Automated retraining pipelines assume ownership over:

- Training data
- Model architecture
- Weight updates
- Deployment lifecycle

However, in this system:

- The evaluated LLMs are black-box API or hosted models
- The pipeline has no access to model weights
- Retraining is neither programmatically possible nor contractually allowed

As a result, retraining cannot be triggered, even if performance degradation is detected.

5. What Is Used Instead of Retraining

Rather than retraining, this project uses:

- Versioned evaluation tracking via DVC
- Automated orchestration via Apache Airflow
- Repeatable batch evaluations
- LLM-based judging
- Bias and failure detection metrics
- Deployment gating based on evaluation outcomes

This ensures that:

- Model regressions are detected
- Unsafe behavior is blocked from deployment
- But no retraining is attempted, since it is outside the control scope of this system.